A Project Report on

# Restaurant Management System

**DEGREE IN**
**MASTER OF COMPUTER APPLICATIONS**

Submitted By

**HARSH VISHWAKARMA**

2400380140047

Under the supervision of

**Prof .Smita Kansal**

GHAZIABAD
(Estd. 1995)
NAAC Accredited A+ Grade Institute
NBA Accredited MCA Programme
www.pg.its.edu.in

**INSTITUTE OF TECHNOLOGY & SCIENCE MOHAN NAGAR, GHAZIABAD**

# STUDENT CERTIFICATE

I hereby certify that the **Mini Project Report titled "Restaurant Management System "** has been completed and prepared by me as a part of the academic requirements of the MCA program during the academic session 2024-2026. .

I further certify that this project is my original work and has not been submitted earlier for any academic or non-academic purpose.

This project supports the following **United Nations Sustainable Development Goals (SDGs)**. I have marked (✓) the SDGs covered in this project:

**SDG Checklist**

| SDG | Description | SDG | Description |
|---|---|---|---|
| ☐ SDG 1 | No Poverty | ☐ SDG 10 | Reduced Inequalities |
| ☐ SDG 2 | Zero Hunger | ☐ SDG 11 | Sustainable Cities & Communities |
| ☐ SDG 3 | Good Health & Well-being | ☐ SDG 12 | Responsible Consumption & Production |
| ☐ SDG 4 | Quality Education | ☐ SDG 13 | Climate Action |
| ☐ SDG 5 | Gender Equality | ☐ SDG 14 | Life Below Water |
| ☐ SDG 6 | Clean Water & Sanitation | ☐ SDG 15 | Life on Land |
| ☐ SDG 7 | Affordable & Clean Energy | ☐ SDG 16 | Peace, Justice & Strong Institutions |
| ☐ SDG 8 | Decent Work & Economic Growth | ☐ SDG 17 | Partnerships for the Goals |
| ☐ SDG 9 | Industry, Innovation & Infrastructure | | |

I have sincerely worked on the research, analysis, and documentation of this project under the guidance of my mentor.

**Student Name:** **HARSH VISHWAKARMA**

**Roll No.:** **2400380140047**

**Class/Program:** **MCA** **Signature:**

## CERTIFICATE (Issued by Project Mentor)

This is to Certify that Harsh Vishwakarma has carried out the project work presented in this report entitled **“Restaurant Management System”** for the award of **MASTER OF COMPUTER APPLICATIONS** from Institute of Technology & Science, Mohan Nagar, Ghaziabad, under my supervision. The report embodies result of original work and studies carried out by the student himself and the contents of the report do not form the basis for the award of any other degree to the candidate or to anybody else.

Date:

Prof. Smita Kansal
Coordinator - MCA
Institute of Technology & Science Mohan Nagar, Ghaziabad

# ONLINE COURSE COMPLETION CERTIFICATE

This is to certify that I, HARSH VISHWARMA Roll No**: 2400380140047** have successfully completed the following online course as part of my project work:

Course Title: PYTHON

Platform: SANFOUNDARY Global Education

Duration: 60 Min.

Completion Date: 14 October 2025

I hereby declare that I have personally completed this course with full sincerity and dedication. The knowledge gained through this course has contributed to the improvement and understanding of my project topic.

Student’s Name: Harsh Vishwakarma.

Student’s Signature: Date:

# Abstract:

Online Food Delivery System is a system which will help restaurant to optimized and control over their restaurants. For the waiters, it is making life easier because they don't have to go kitchen and give the orders to chef easily. For the management point of view, the manager will able to control the restaurant by having all the reports to hand and able to see the records of each employees and orders. This website helps the restaurants to do all functionalities more accurately and enhances the spend of all the tasks. Online Food Delivery System reduces manual works and improves efficiency of restaurant. The online food delivery system set up menu online and the customers easily places the order with a simple mouse click. Also with a food menu online you can easily track the orders, maintain customer's database and improve your food delivery service. This system allows the user to select the desired food items from the displayed menu. The user orders the food items. The payment can be made online or pay-on-delivery system. The user's details are maintained confidential because it maintains a separate account for each user. An id and password is provided for each user. Therefore, it provides a more secured ordering.

# ACKNOWLEDGEMENT

With great pleasure, I take this opportunity to express my sincere gratitude to all those who contributed towards the successful completion of my MCA project titled **“Restaurant Management System.”**

I would like to extend my deepest appreciation to my mentor **Prof Smita Kansal**, whose expert guidance, motivation, and constant encouragement greatly helped me in completing this project successfully.

I am also grateful to the **Department of Computer Science and the faculty members of Institute Of Technology And Science Mohan Nagar Ghaziabad** for providing a supportive learning environment and necessary facilities required to complete this project.

Lastly, I extend my thanks to my classmates, family, and friends for their continuous support and inspiration throughout the development of this project.

**HARSH VISHWAKARMA**

**MCA ( II nd Year)**

# Introduction:

**CHAPTER:1**

## Brief Introduction:

Smart Restaurant Management System is a new generation of restaurant management software. When users/customer will enter in the website, he/she should have an account. If user does not have an account, user has to create a new account to order food. To create a new account user should enter unique username, email and new mobile no. with password. User fills his/her address for food delivery. Once user enters in the website, you can see different types of food available in restaurant. First select category of food from soups, starters, the main course dishes and desserts. After that search food as your interest. select food you want to order. After selecting all your meal place your order and confirm your address. Then website will saw you various type of payment methods and your total bill amount. You can pay cash on delivery or there are many more options for online payment to get benefits. online payment methods:

- Credit/Debit card payments
- Bank transfers
- E-Wallets
- UPI payments

You can choose best deal for your meal.

## Tools/Technologies Used:

**Technologies:**

- Jingo
- Python
- Bootstrap
- HTML
- CSS

**Tools:**

- Get
- Visual Studio Code
- Pharm

# Software Requirement Specifications:

## Users of the System:

- Three types of users should be able to use the system: **customer**, **employee** and **administrator**.
- Customers are users who visit the website and can create orders by customizing food, selecting products and entering customer details.
- Employees are the group of users that work with the ordering system on a daily basis. Employees will have their own accounts to log on to. They are the ones responsible for processing orders.
- The administrator, or super user, has the ultimate control of the system, he can add, change or delete ingredients and products, as well as add, change, or delete employee accounts.

## Functional Requirements:

### Customers:

1. Sign Up (only for new customer)
   **Input**: "SignUp" option selected.
   **Output**: customer prompted to enter the details.

2. Login
   **Input**: "Login" option selected.
   **Output**: customer prompted to enter the username and password.

3. Forgot password
   **Input**: "forgot password" option selected.
   **Output**: customer prompted to enter the email and new password.

4. Select food items
   **State**: The customer has logged in and the main menu has been displayed.
   **Input**: Items are selected customer feel free to order.
   **Output**: System will display selected items.

5. Changes to order
   **Input**: “go to cart” option selected.
   **Output**: customer can delete or add food item in order.

6. Review the order before submitting
   **Input**: “Order Place” option selected.
   **State**: Customer name, phone number, location (address) display or enter the all of information.
   **Output**: customer prompted to pay the bill.

7. Payment
   **State**: The different types of payment method are display.
   **Input**: choose any payment method.
   **Output**: customer prompted to enter the verification code if choose online payment.
   **State**: Display order no., payment details and confirmation of delivery.

8. Logout
   **Input**: “Logout” option selected.
   **Output**: you are successfully logout.
   **State**: System display login page.

## Employees:

1. Login (Employee login page)
   **Input**: “Login” option selected.
   **Output**: Employee prompted to enter the username and password.

2. Modify Menu
   **State**: In the system all the items are displayed with their rates.
   **Input**: “Change” option selected.
   **Output**: Employee can make changings in menu like adding or removing food items which are not available and changings rate of items.

3. Order list
   **Input**: “Order list” option selected.
   **State**: System display all order details.
   **Output**: Employee can make changings like confirm order, prepared order, delivered order, not confirm order.

4. Logout
   **Input**: “Logout” option selected.
   **Output**: you are successfully logout.
   **State**: System display login page.

## Administrators:

- Administrators must be able to use Employees all features.

  1. Login (admin login page)
     **Input**: “Login” option selected.
     **Output**: admin prompted to enter the username and password.

  2. Logout
     **Input**: “Logout” option selected.
     **Output**: you are successfully logout.
     **State**: System display login page.

# Non-Functional Requirements:

1. **Portability**
   System running on one platform can easily be converted to run on another platform.

2. **Reliability**
   The ability of the system to behave consistently in a user-acceptable manner when operating within the environment for which the system was intended.

3. **Availability**
   The system should be available at all times, meaning the user can access it using a web browser, only restricted by the down time of the server on which the system runs.

4. **Maintainability**
   A commercial database is used for maintaining the database and the application server takes care of the site.

5. **Security**
   Secure access of confidential data (customer information).

6. **User friendly**
   System should be easily used by the customer.

7. **Performance**
   Performance should be fast.

8. **Efficient**
   System should be efficient that it won’t get hang if heavy traffic of order is placed.

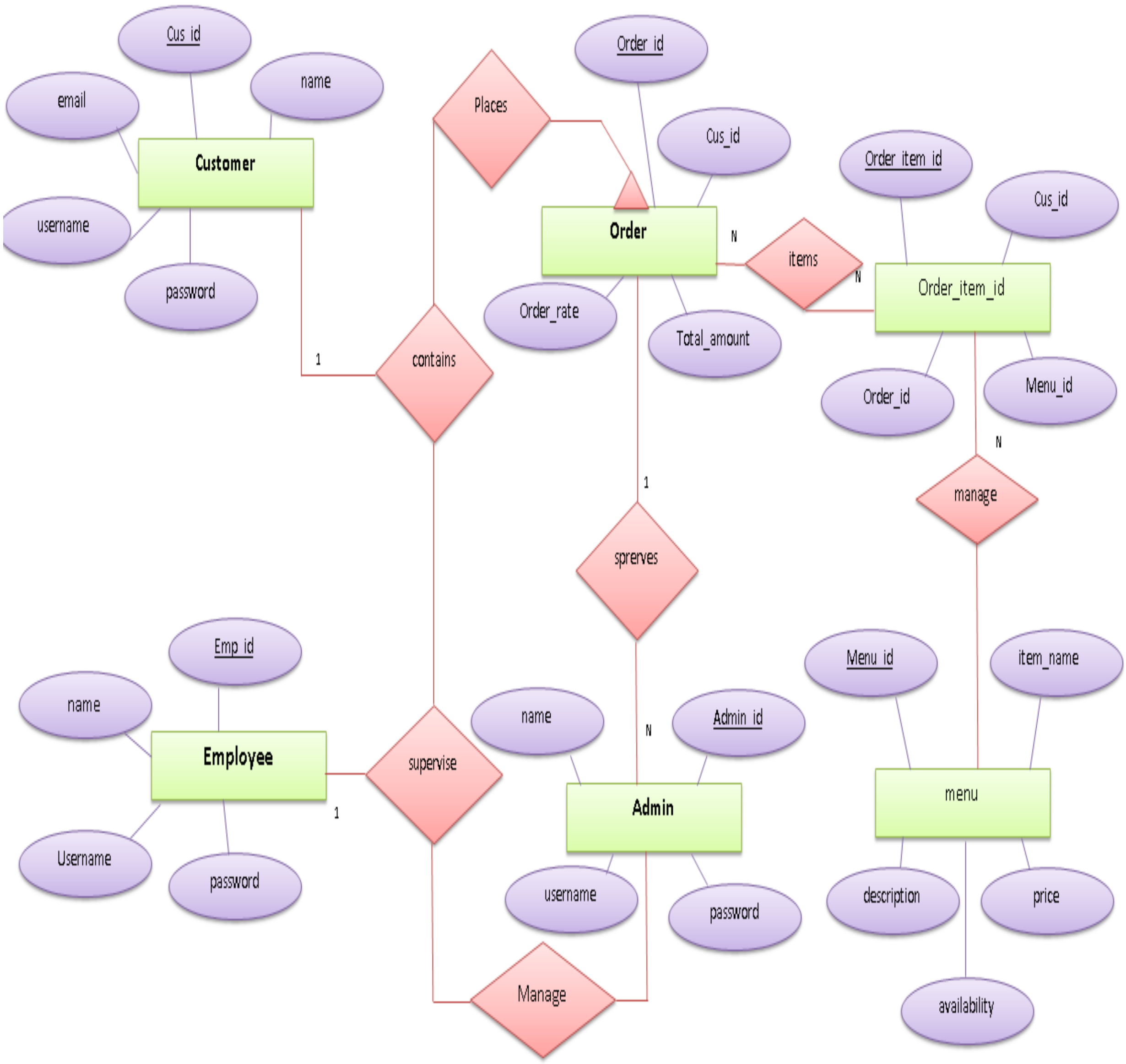
9. **Safety**
   Data in the database of system should not loss or damage.

10. **Privacy**
    Personal data of the system should not disclose to anyone.

# Design Documents:

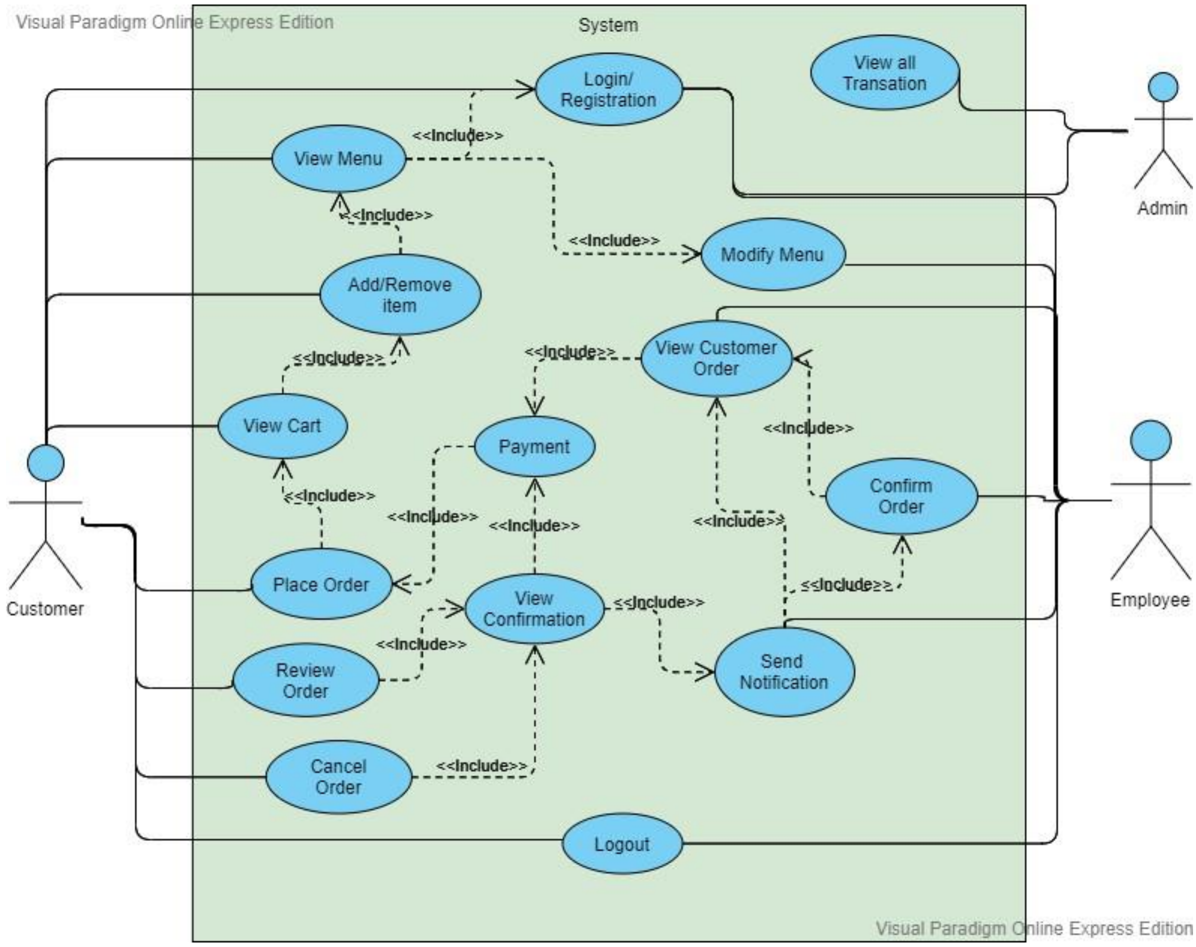
**CHAPTE4R:2**

## ER Diagram:



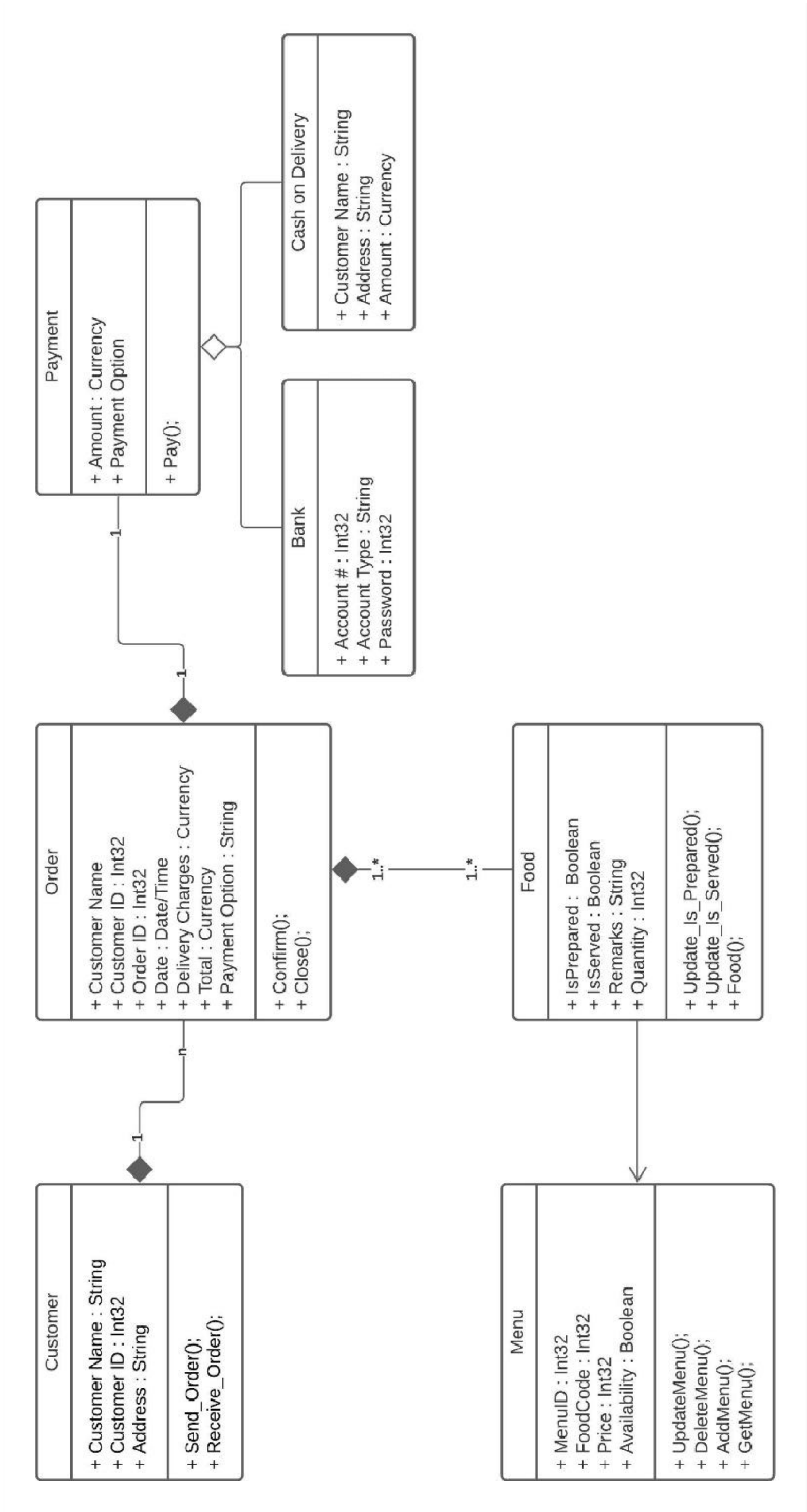
# Use Case Diagram:

- Administrators(admin) can be able to use Employees all features.

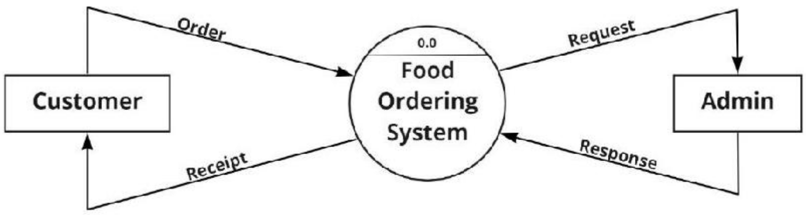



- In this use case diagram proposed have 3 actors, namely Customer, Employee and Admin.
- Customers can access the website to place orders and payments while Employee and Admin can access the website to receive orders, payments and making order report.
- Use case starts from login into the website with the customer's username and password, ordering food, making payment. Then Employee makes an order report based on customer order data.

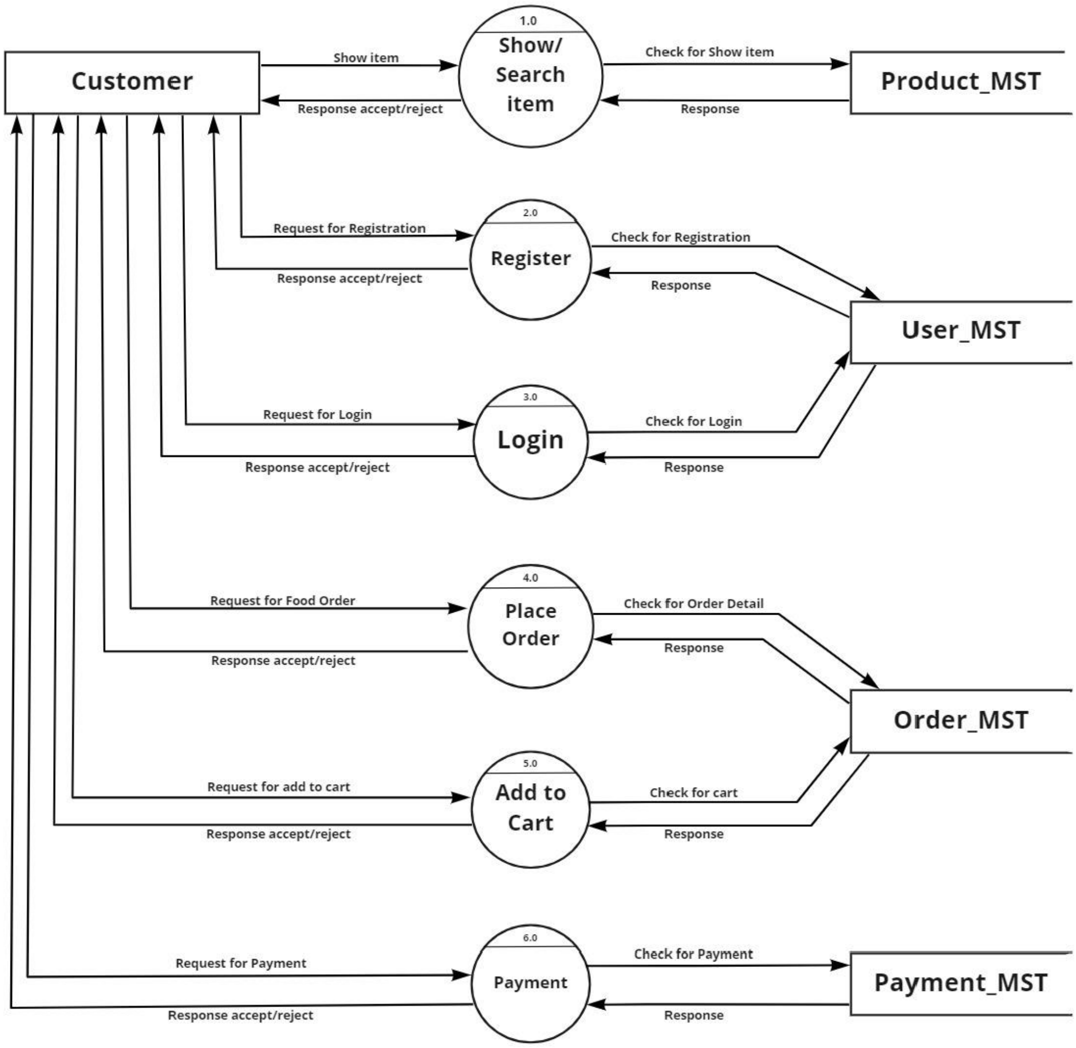
# Class Diagram:



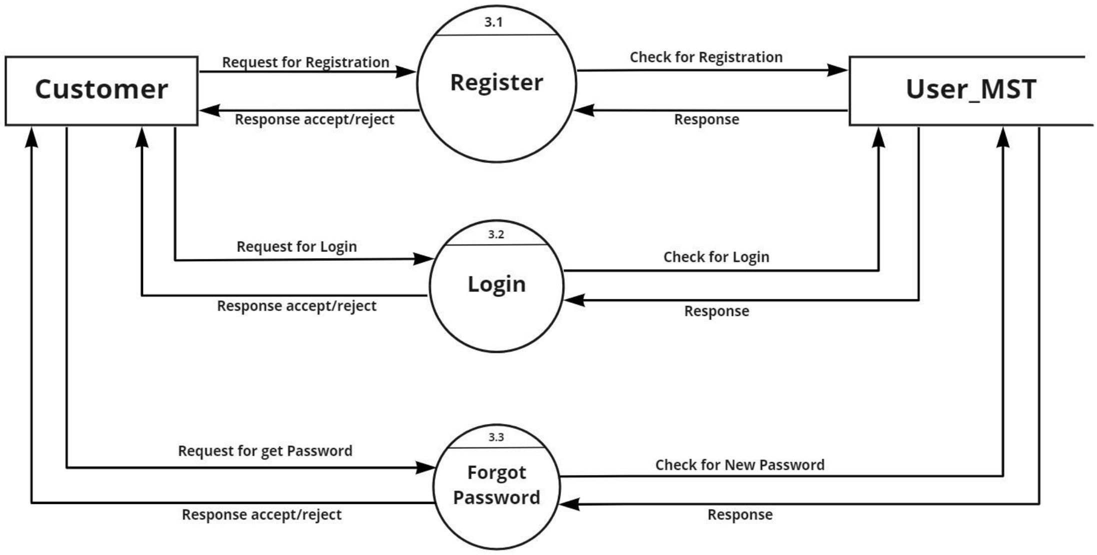
# DFD Model:




Level 0 DFD

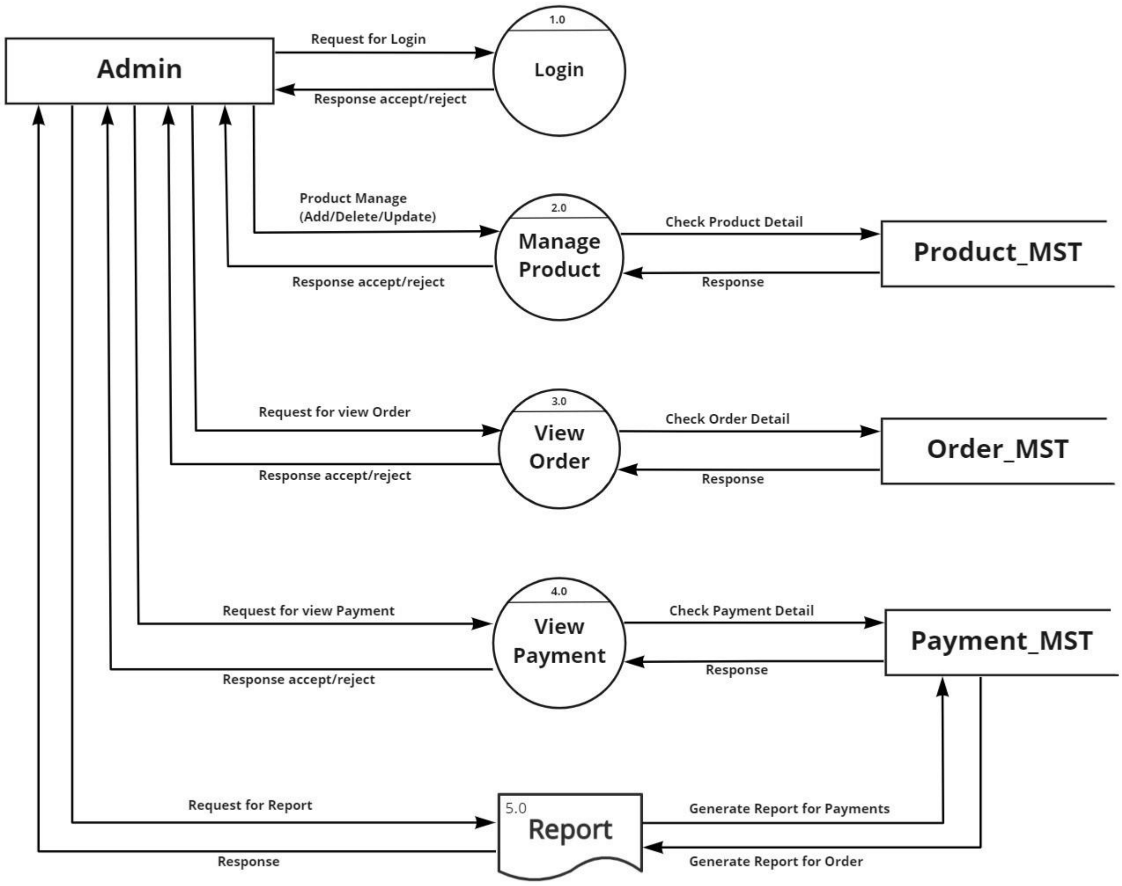



Level 1 DFD



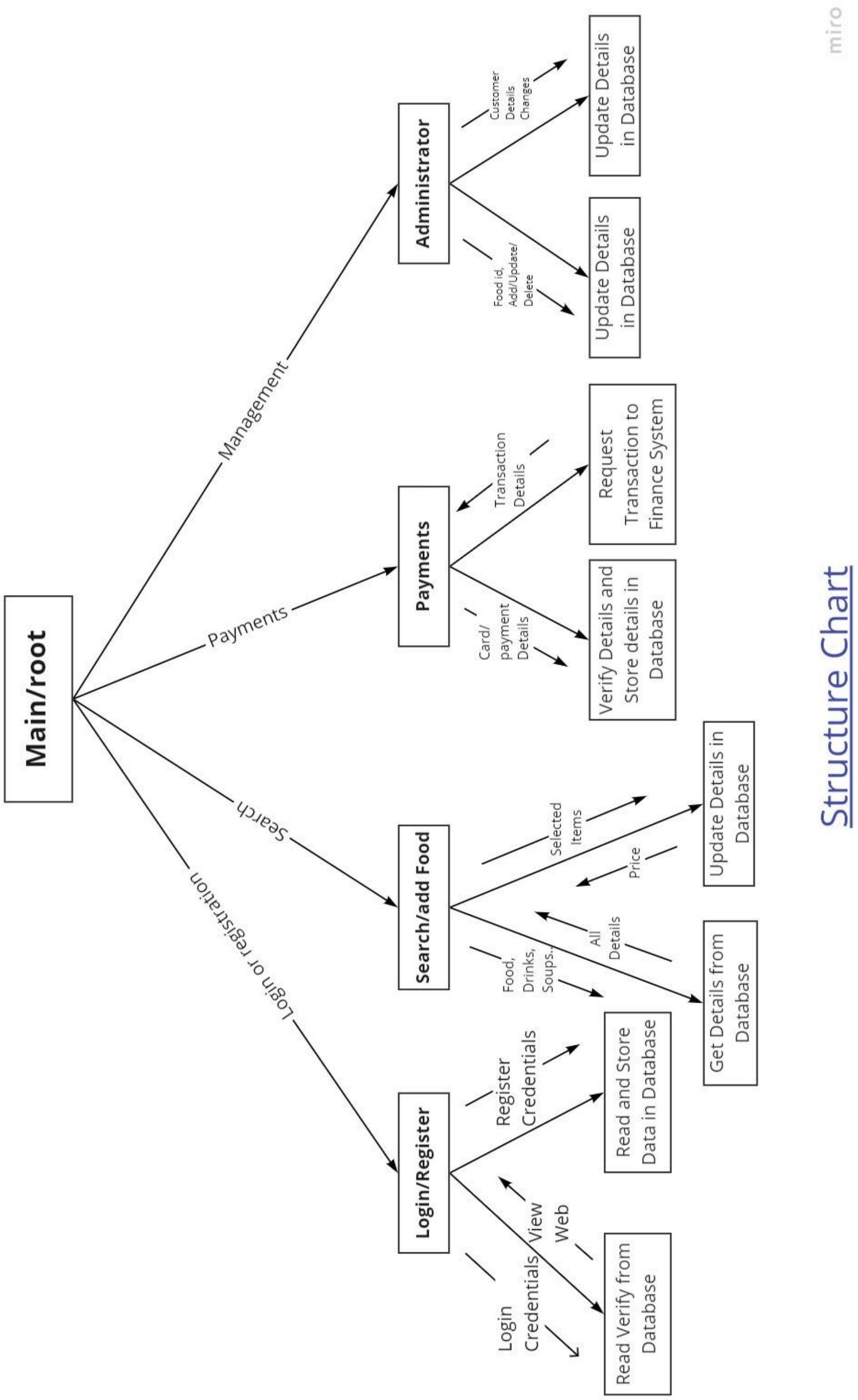

**Level 2 DFD(3.0)**



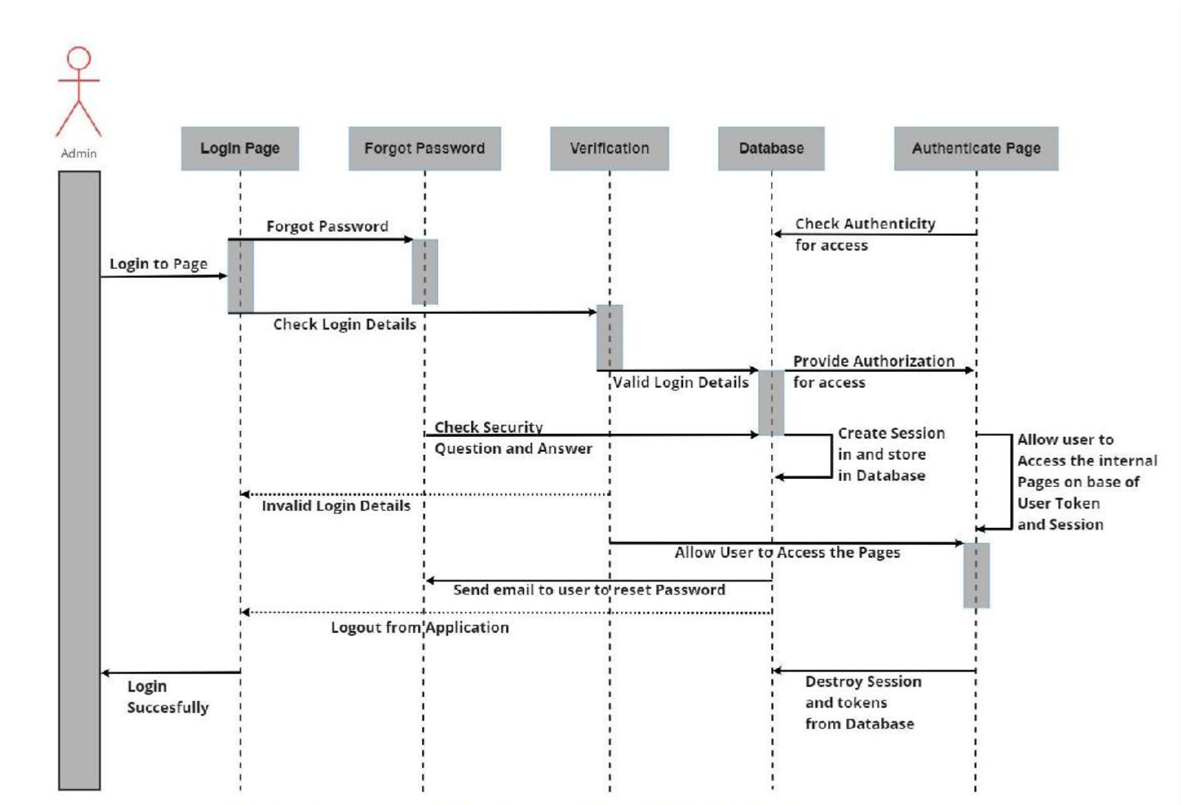



**Admin side DFD**

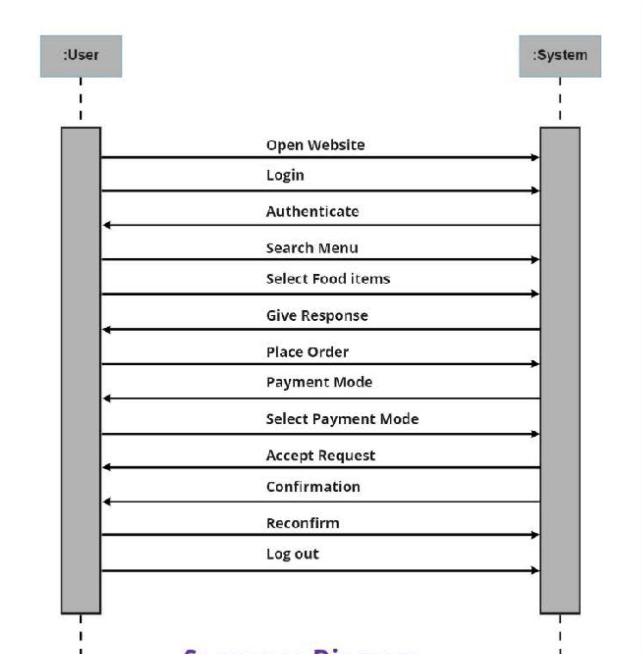
## Structure Chart:



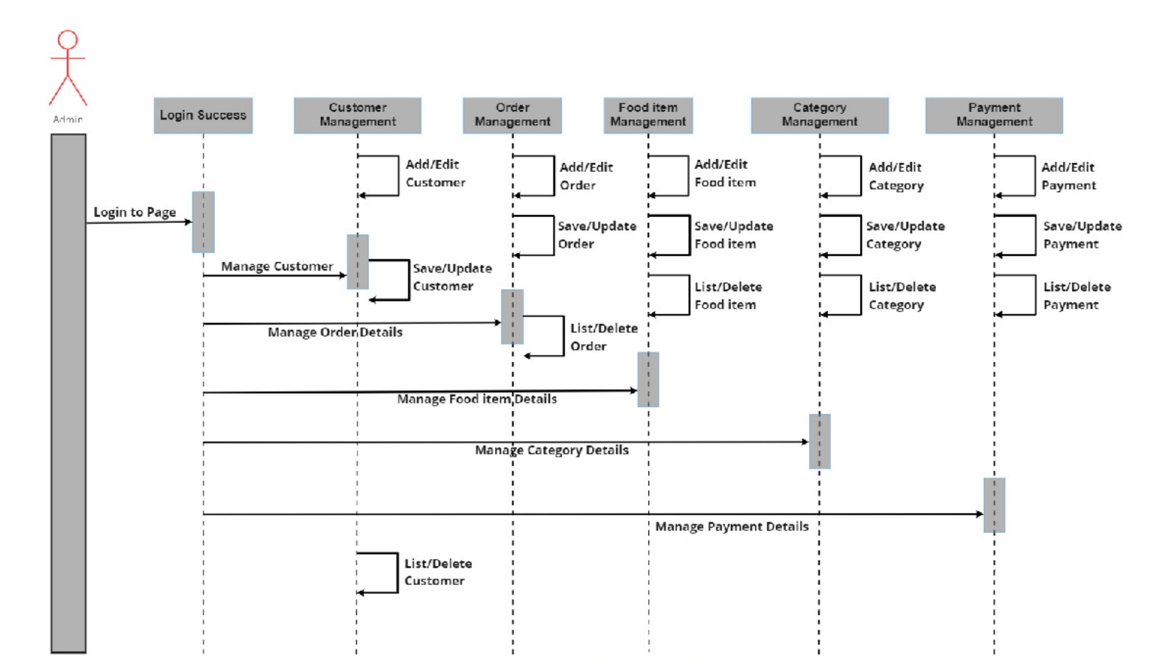
## Sequence Diagrams:




Login Sequence Diagram of Food Order System

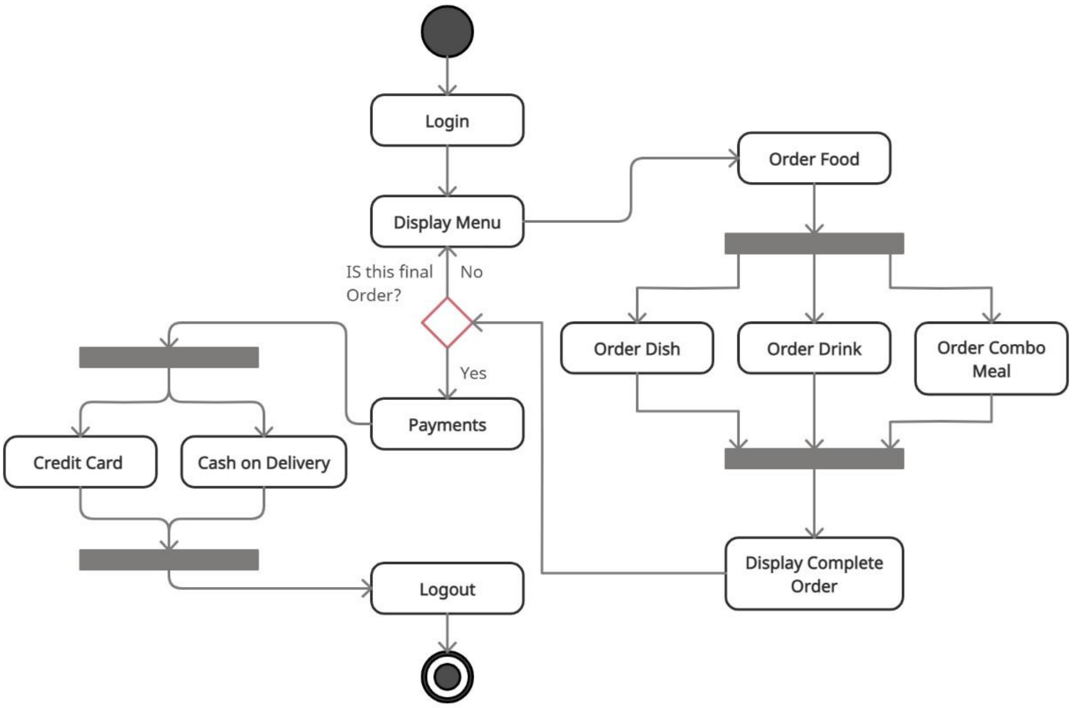



Sequence Diagram

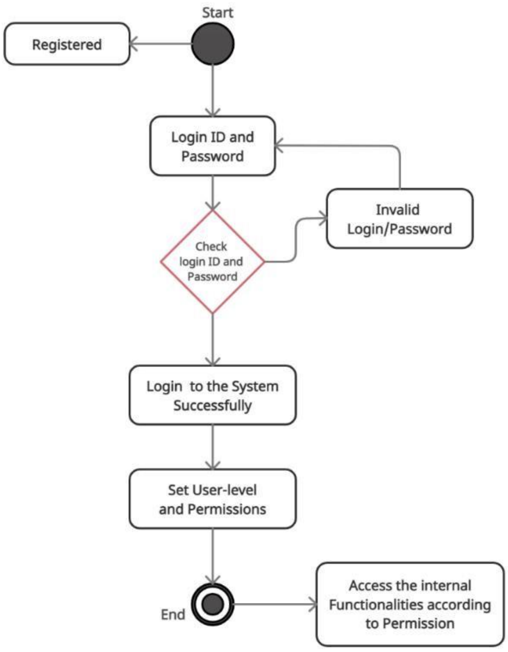



UML Sequence Diagram of Food Order System

# Activity Diagrams:




Activity Diagram




Login Activity Diagram

# Implements Details:

**CHAPTER:3**

## Modules

- ❖ The system consists of 4 basic modules namely

  1. User Module
  2. Product Module
  3. Order Module
  4. Order Status Update Module

- ➢ Implementation is done using Jingo.

### ❖ User Module

The main aim of the User Module is provide all the functionality related users. It tracks all the information of the customers. We have developed all type of operations of the customers. This is role based Module Where Admin can perform each and every operations on data but customer only view his/her data, so access level restrictions has also been implemented on the project.

### ❖ Product Module

The main purpose for developing the Product Module is to manage Products category wise. All product will be managed by admin and Customer will be able to see product and buy them. Admin can see the list, change product details and also add or delete products.

### ❖ Order Module

The main aim of the Order Module is receive all order details and display them. It is designed to be used only by restaurant employees (and admin), and provides the following functions: Retrieve new orders from the database and Display the orders in an easily readable, graphical way. Under “View Order” a customer will be able to see only his/her order.

### ❖ Order Status Update Module

The main aim of this Module is update all information related to order. Admin or employee can change or add order status. Customer only see his/her order status details. Under “Tracker” a customer will be able to see his/her order all status details.

# Function prototypes

## Login Page:

```
    }
{% endblock %}

  {% block body %}
  {% load static %}

<div class="container">
    <!--Slideshow starts here -->
    {% for product, range, nSlides in allProds %}
    <h3 class="my-4">{{product.0.category}}</h3>
    <div class="row">
        <div id="demo{{forloop.counter}}" class="col carousel slide my-3" data-ride="carousel">
            <ul class="carousel-indicators">
                <li data-target="#demo{{forloop.counter}}" data-slide-to="0" class="active"></li>
                {% for i in range %}
                <li data-target="#demo{{forloop.parentloop.counter}}" data-slide-to="{{i}}"></li>
                {% endfor %}
            </ul>
            <div class="container carousel-inner no-padding">
                <div class="carousel-item active">
                    {% for i in product %}
                    <div class="col-xs-3 col-sm-3 col-md-3">
                        <div class="card align-items-center" style="width: 18rem;">
                            <img src='/media/{{i.image}}' class="card-img-top" alt="...">
                            <div class="card-body">
                                <h5 class="card-title" id="namepr{{i.id}}">{{i.product_name|slice:"0:20"}}...</h5>
                                <h6 style="color: #ff0000">Rs. <i id="pricepr{{i.id}}">{{i.price}}</i>/-</h6>
                                <p class="card-text">{{i.desc|slice:"0:23"}}...</p>
                                <span id="divpr{{i.id}}" class="divpr">
                                    <button id="pr{{i.id}}" class="btn btn-primary cart">Add to Cart</button>
                                </span>
                                <a href="/shop/productView/{{i.id}}"> <button id="qv{{i.id}}" class="btn btn-primary c
                            </div>
                        </div>
                    </div>
                    {% if forloop.counter|divisibleby:4 and forloop.counter > 0 and not forloop.last %}
```

## Inedx page :


```
    }
{% endblock %}

  {% block body %}
  {% load static %}

<div class="container">
    <!--Slideshow starts here -->
    {% for product, range, nSlides in allProds %}
    <h3 class="my-4">{{product.0.category}}</h3>
    <div class="row">
        <div id="demo{{forloop.counter}}" class="col carousel slide my-3" data-ride="carousel">
            <ul class="carousel-indicators">
                <li data-target="#demo{{forloop.counter}}" data-slide-to="0" class="active"></li>
                {% for i in range %}
                <li data-target="#demo{{forloop.parentloop.counter}}" data-slide-to="{{i}}"></li>
                {% endfor %}
            </ul>
            <div class="container carousel-inner no-padding">
                <div class="carousel-item active">
                    {% for i in product %}
                    <div class="col-xs-3 col-sm-3 col-md-3">
                        <div class="card align-items-center" style="width: 18rem;">
                            <img src='/media/{{i.image}}' class="card-img-top" alt="...">
                            <div class="card-body">
                                <h5 class="card-title" id="namepr{{i.id}}">{{i.product_name|slice:"0:20"}}...</h5>
                                <h6 style="color: #ff0000">Rs. <i id="pricepr{{i.id}}">{{i.price}}</i>/-</h6>
                                <p class="card-text">{{i.desc|slice:"0:23"}}...</p>
                                <span id="divpr{{i.id}}" class="divpr">
                                    <button id="pr{{i.id}}" class="btn btn-primary cart">Add to Cart</button>
                                </span>
                                <a href="/shop/productView/{{i.id}}"> <button id="qv{{i.id}}" class="btn btn-primary c
                            </div>
                        </div>
                    </div>
                    {% if forloop.counter|divisibleby:4 and forloop.counter > 0 and not forloop.last %}
```

## Checkout

```
def checkout(request):
    if request.method == "POST":
        items_json = request.POST.get('itemsJson', '')
        user_id = request.POST.get('user_id', '')
        name = request.POST.get('name', '')
        amount = request.POST.get('amount', '')
        email = request.POST.get('email', '')
        address = request.POST.get('address1', '') + " " + request.POST.get('address2', '')
        city = request.POST.get('city', '')
        state = request.POST.get('state', '')
        zip_code = request.POST.get('zip_code', '')
        phone = request.POST.get('phone', '')
        order = Orders(items_json=items_json, userId=user_id, name=name, email=email, address=address,
        city=city, state=state, zip_code=zip_code, phone=phone, amount=amount)
        order.save()
        update = OrderUpdate(order_id=order.order_id, update_desc="The Order has been Placed")
        update.save()
        thank = True
        id = order.order_id
        if 'onlinePay' in request.POST:
            # Request paytm to transfer the amount to your account after payment by user
            darshan_dict = {

                'MID': 'WorldP64425807474247',  # Your-Merchant-Id-Here
                'ORDER_ID': str(order.order_id),
                'TXN_AMOUNT': str(amount),
                'CUST_ID': email,
                'INDUSTRY_TYPE_ID': 'Retail',
                'WEBSITE': 'WEBSTAGING',
                'CHANNEL_ID': 'WEB',
                'CALLBACK_URL': 'http://127.0.0.1:8000/shop/handlerequest/',

            }
            darshan_dict['CHECKSUMHASH'] = Checksum.generate_checksum(darshan_dict, MERCHANT_KEY)
            return render(request, 'shop/paytm.html', {'darshan_dict': darshan_dict})
        elif 'cashOnDelivery' in request.POST:
            return render(request, 'shop/checkout.html', {'thank': thank, 'id': id})
    return render(request, 'shop/checkout.html')
```

## View all Products

```python
def index(request):
    allProds = []
    catprods = Product.objects.values('category', 'id')
    cats = {item['category'] for item in catprods}
    for cat in cats:
        prod = Product.objects.filter(category=cat)
        n = len(prod)
        nSlides = n // 4 + ceil((n / 4) - (n // 4))
        allProds.append([prod, range(1, nSlides), nSlides])
    darshan = {'allProds': allProds}
    return render(request, 'shop/index.html', darshan)
```

## Track Order

```python
def tracker(request):
    if request.method == "POST":
        orderId = request.POST.get('orderId', '')
        email = request.POST.get('email', '')
        name = request.POST.get('name', '')
        password = request.POST.get('password')
        user = authenticate(username=name, password=password)
        if user is not None:
            try:
                order = Orders.objects.filter(order_id=orderId, email=email)
                if len(order) > 0:
                    update = OrderUpdate.objects.filter(order_id=orderId)
                    updates = []
                    for item in update:
                        updates.append({'text': item.update_desc, 'time': item.timestamp})
                        response = json.dumps({"status": "success", "updates": updates, "itemsJson": order[0].items_json}, default=str)
                    return HttpResponse(response)
                else:
                    return HttpResponse('{"status":"noitem"}')
            except Exception as e:
                return HttpResponse('{"status":"error"}')
        else:
            return HttpResponse('{"status":"Invalid"}')
    return render(request, 'shop/tracker.html')
```

## Order List

```python
def orderView(request):
    if request.user.is_authenticated:
        current_user = request.user
        orderHistory = Orders.objects.filter(userId=current_user.id)
        if len(orderHistory) == 0:
            messages.info(request, "You have not ordered.")
            return render(request, 'shop/orderView.html')
        return render(request, 'shop/orderView.html', {'orderHistory': orderHistory})
    return render(request, 'shop/orderView.html')
```

## Contact Admin

```python
def contact(request):
    thank = False
    if request.method == "POST":
        name = request.POST.get('name', '')
        email = request.POST.get('email', '')
        phone = request.POST.get('phone', '')
        desc = request.POST.get('desc', '')
        contact = Contact(name=name, email=email, phone=phone, desc=desc)
        contact.save()
        thank = True
        return render(request, 'shop/contact.html', {'thank': thank})
    return render(request, 'shop/contact.html', {'thank': thank})
```

### Payment Status:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Paytm merchant payment page</title>
</head>
<body>
<h1>Redirecting you to the merchant....</h1>
<h1>Please do not refresh your page....</h1>
<form action="https://securegw-stage.paytm.in/theia/processTransaction" method="post" name="paytm">
    {% for key, value in darshan_dict.items %}
    <input type="hidden" name="{{key}}" value="{{value}}">
    {% endfor %}

</form>
</body>
<script>
document.paytm.submit()
</script>
</html>
```

### Sign Up page:

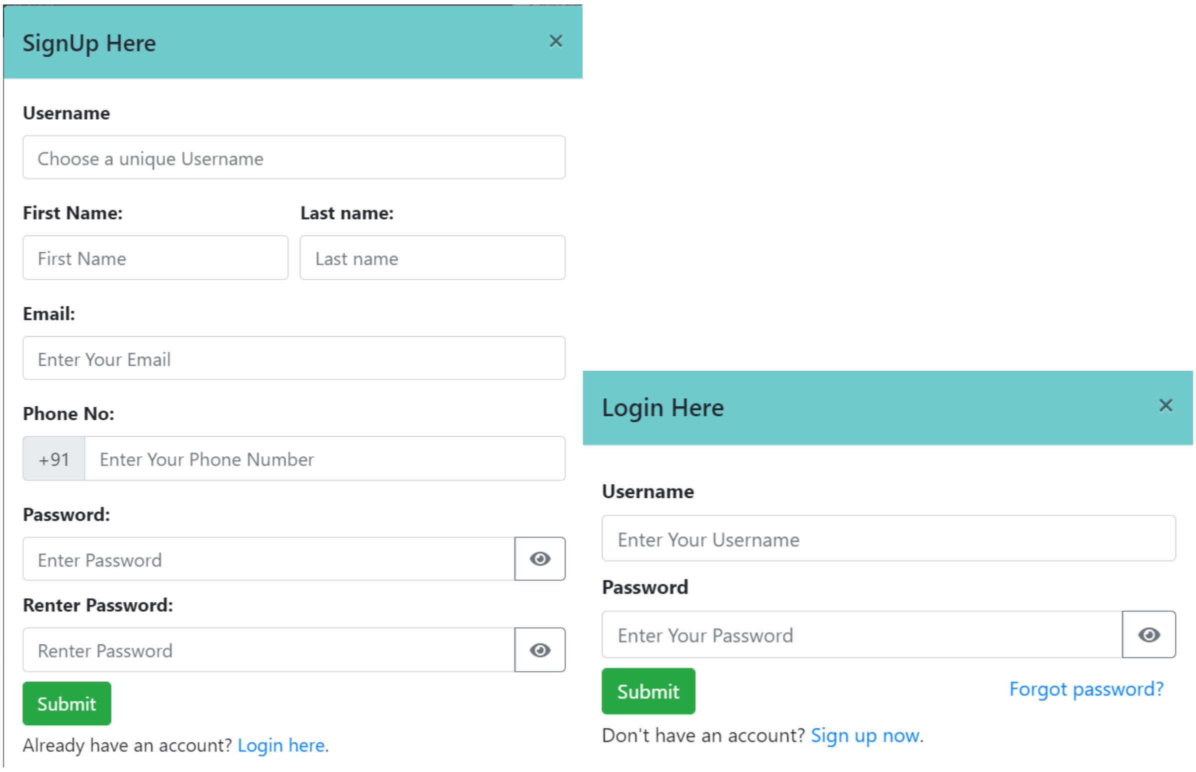
```html
div class="container mt-3 " xmlns="http://www.w3.org/1999/html" xmlns="http://www.w3.org/1999/html">
        <div class="col-md-12 col-lg-6 login-right">

            <div class="login-header">

                <h3 class="jumbotron-heading"><span id="typed1">New Customer! Register Here</span></h3>
            </div>
            <!-- Regis ter Form -->
            <div class="form-group mt-3">
                <form action="/accounts/signup" method="post">
                    {% csrf_token %}
                    <div class="text-left my-2">
                        <label for="username">Username</label>
                        <input class="form-control" id="username" name="username" placeholder="Choose a unique Usernam
                    </div>
                    <div class="form-group">
                        <div class="text-left my-2">
                            <label for="first_name">First Name</label>
                            <input class="form-control" id="first_name" name="first_name" placeholder="Enter Your Firs
                                   type="text">
                        </div>
                         <div class="form-group">
                        <div class="text-left my-2">
                            <label for="last_name">Last Name</label>
                            <input class="form-control" id="last_name" name="last_name" placeholder="Enter Your Last N
                                   type="text">
                        </div>
                            <div class="form-group">
                        <div class="text-left my-2">
                            <label for="email1">Email</label>
                            <input class="form-control" id="email1" name="email1" placeholder="Enter Your Email Addres
                                   type="text">
```

## Contact Page:

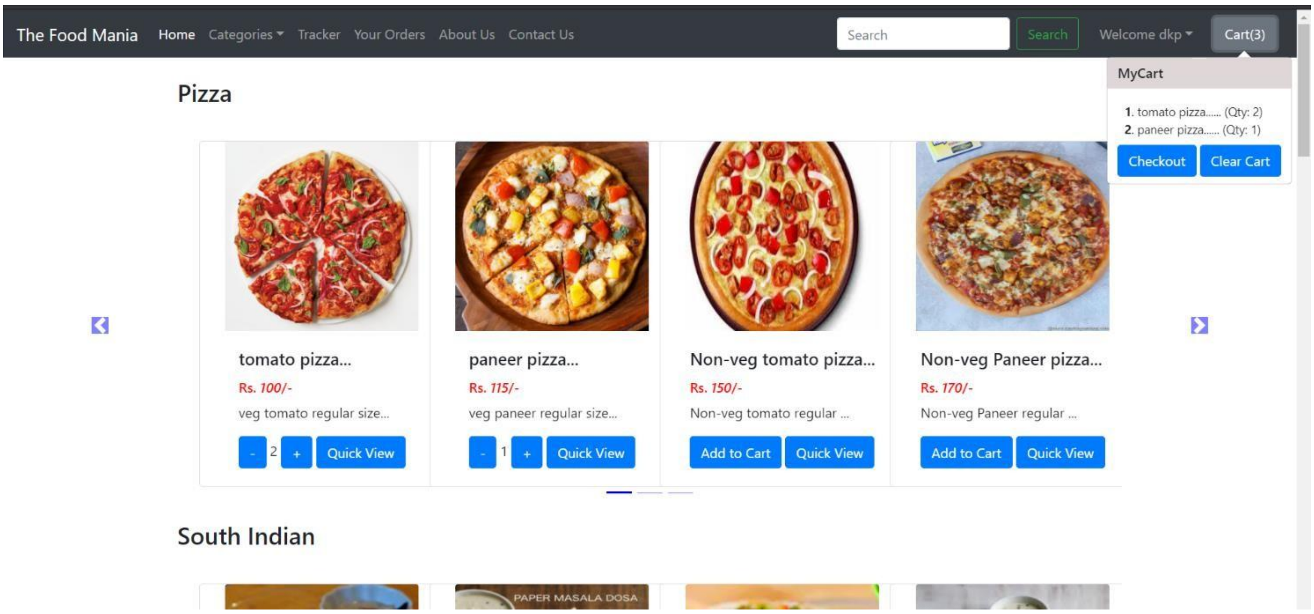
```
<div class="col-lg-12">
  <div class="card card-shadow border-0">
    <div class="row">
      <div class="col-lg-8">
        <div class="contact-box p-4">
          <div class="row">
            <div class="col-lg-8">
              <h4 class="title">Contact Us</h4>
            </div>
          </div>
          <form action="/shop/contact/" method="POST">{% csrf_token %}
            <div class="form-group mt-3">
                <b><label for="name">Name:</label></b>
                <input type="text" class="form-control" id="name" name="name" placeholder="Enter Your Name" requ
            </div>
            <div class="form-group mt-3">
                <b><label for="email">Email:</label></b>
                <input type="email" class="form-control" id="email" name="email" placeholder="Enter Your Email"
            </div>
            <div class="form-group mt-3">
                <b><label for="phone">Phone No:</label></b>
                <div class="input-group mb-3">
                    <div class="input-group-prepend">
                      <span class="input-group-text" id="basic-addon">+91</span>
                    </div>
                    <input type="tel" class="form-control" id="phone" name="phone" aria-describedby="basic-addon
                </div>
            </div>
            <div class="form-group mt-3">
              <b><label for="desc">Message:</label></b>
              <textarea class="form-control" id="desc" name="desc" rows="2" required minlength="6" placeholder="
            </div>
              <button type="submit" class="btn btn-danger-gradiant text-white border-0 py-2 px-3"><span> SUBMIT
          </form>
        </div>
      </div>
```

# Work Flow/Layouts:

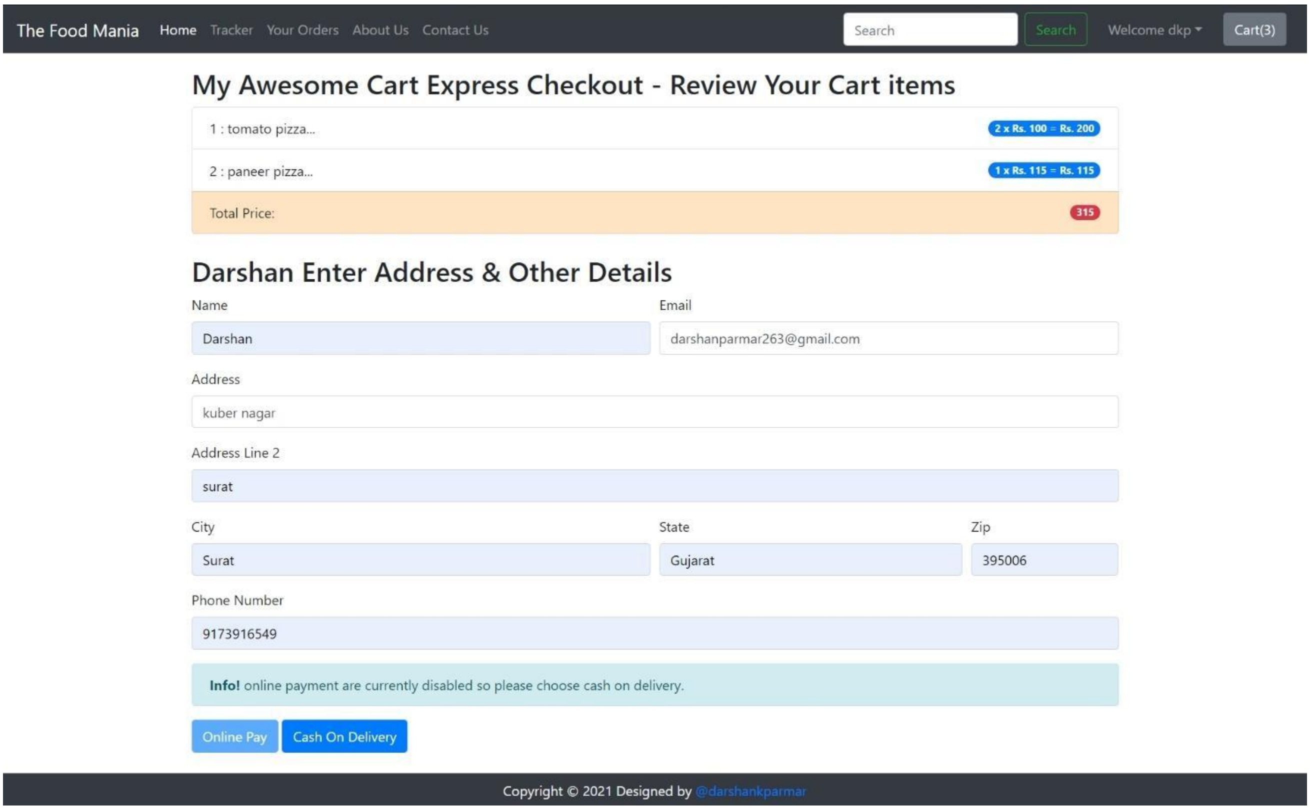
## CHAPTER:4



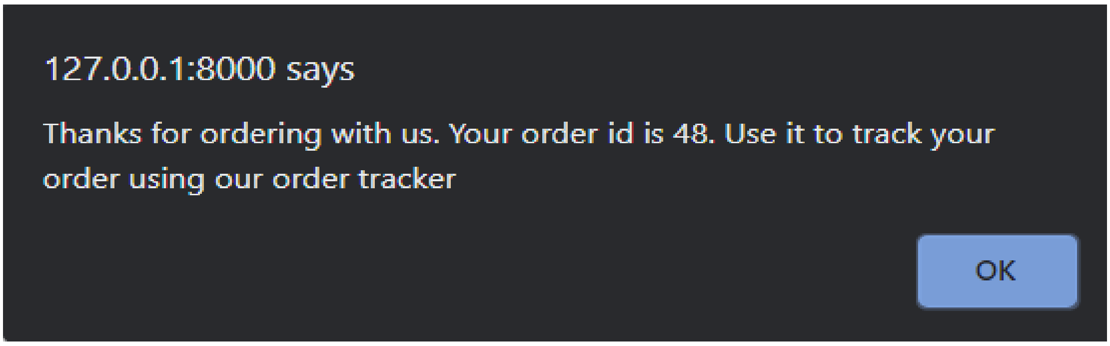

**Login & SignUp module**



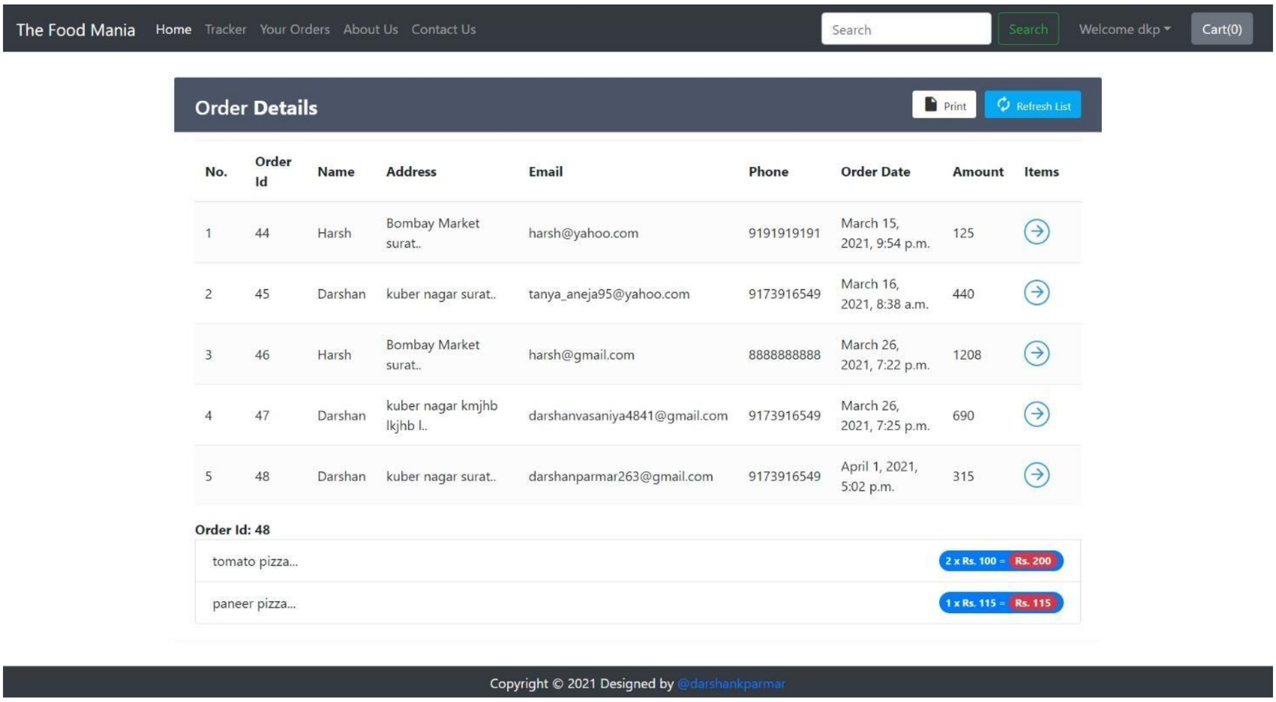

**Home page**



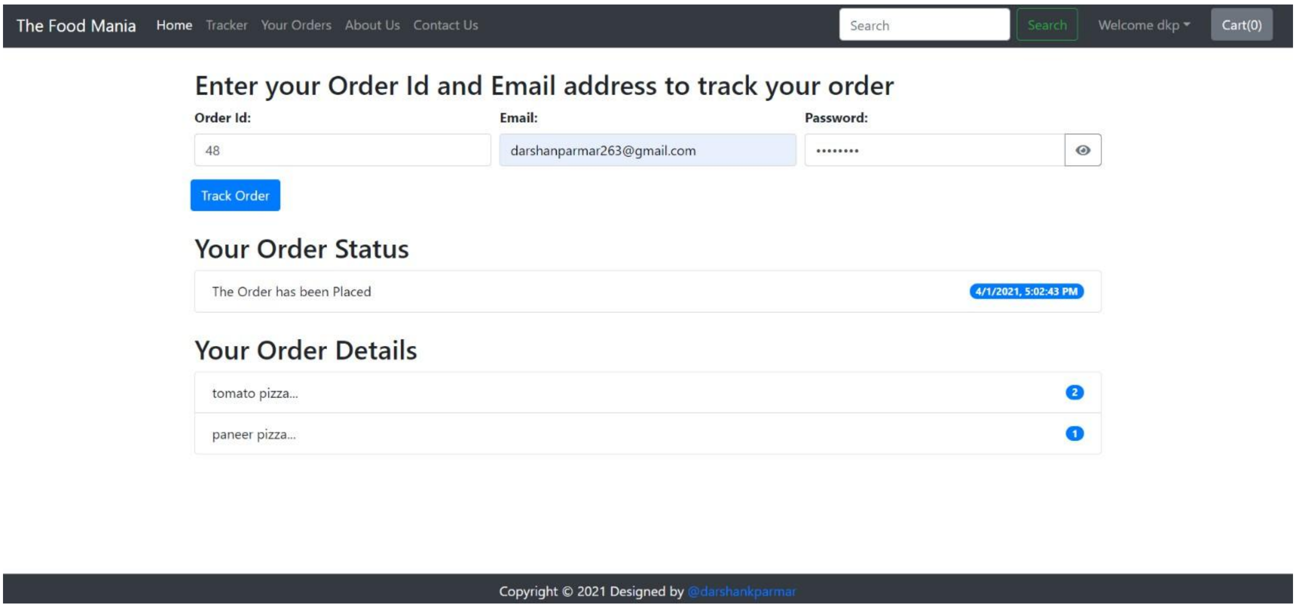

**Chekout Page**



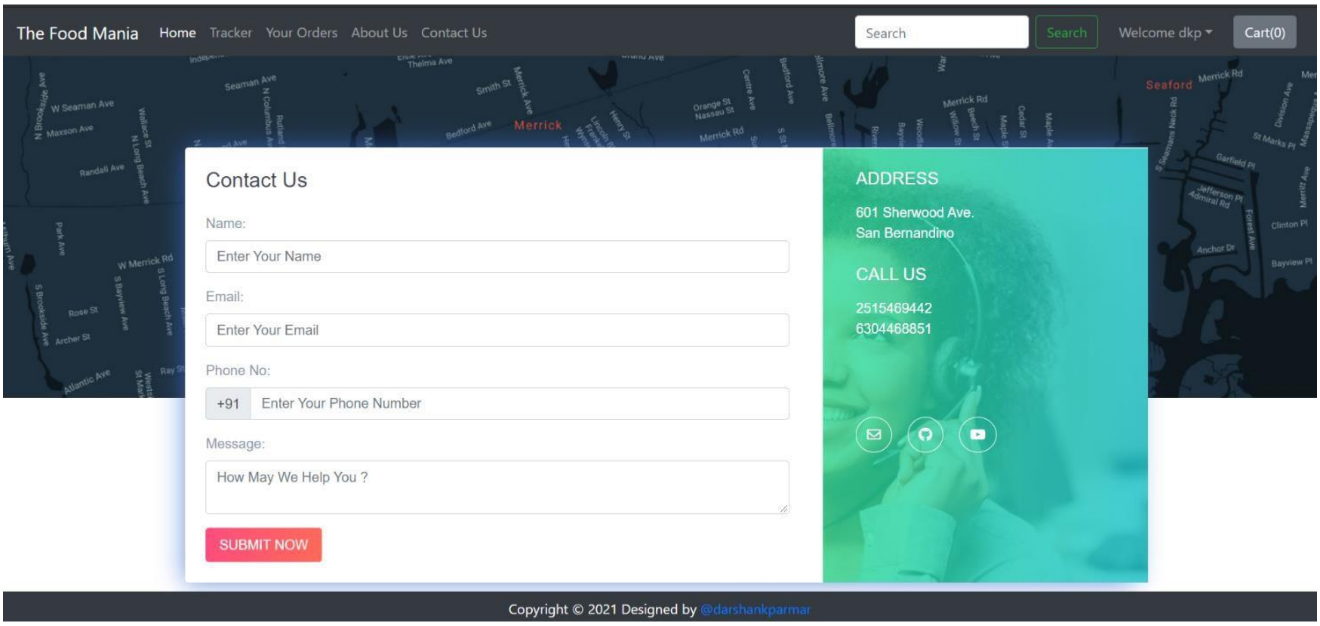

**Confirm Order**



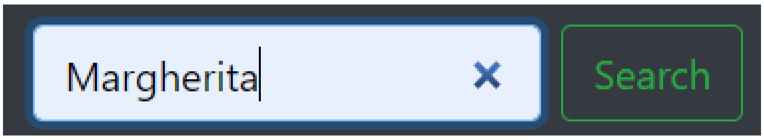

# View Order Page



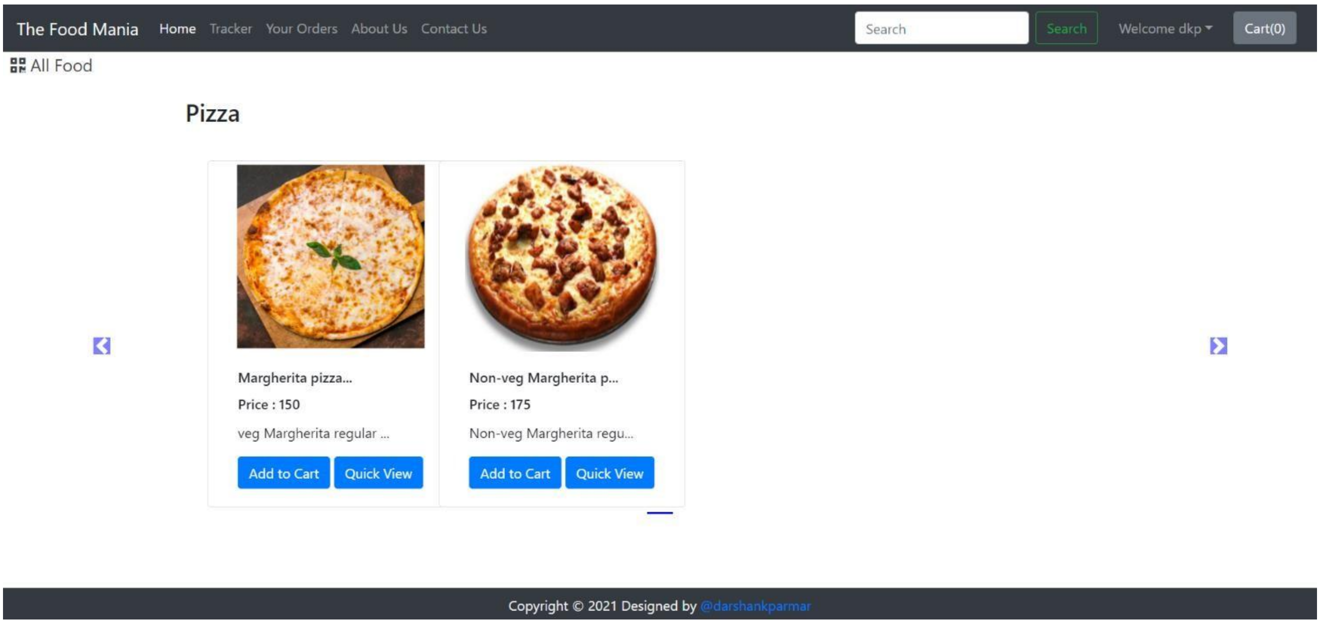

# Tracker Page




# Contact Us Page

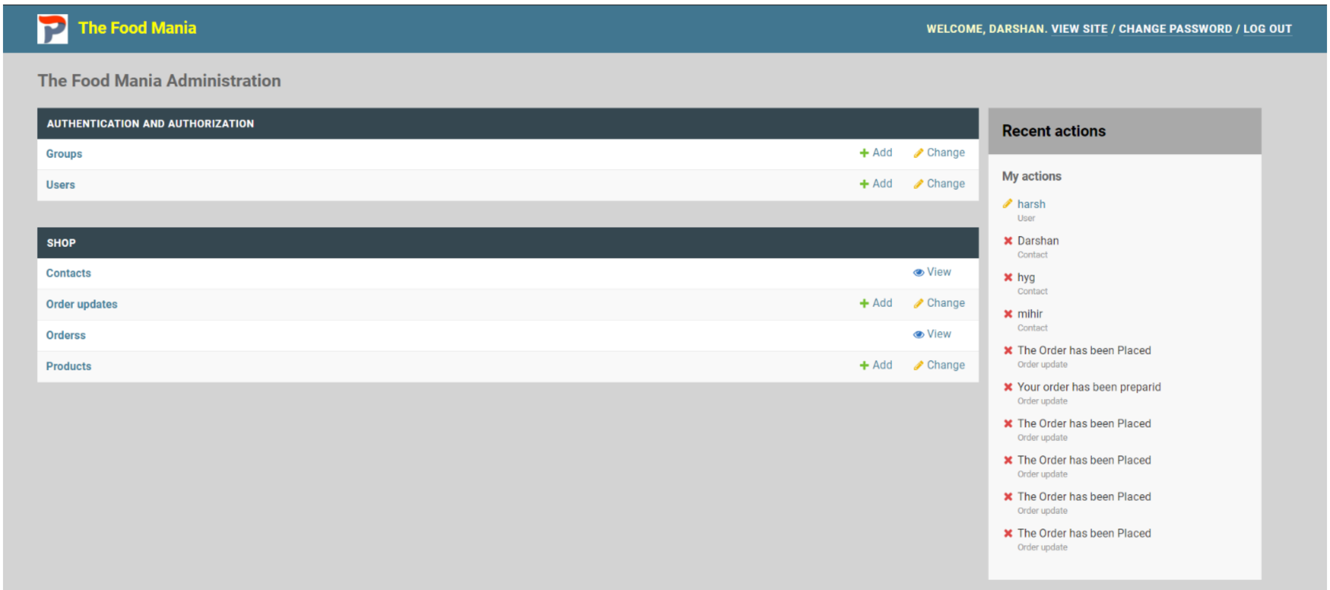



# Search bar




# Search Page


# Admin Login Page

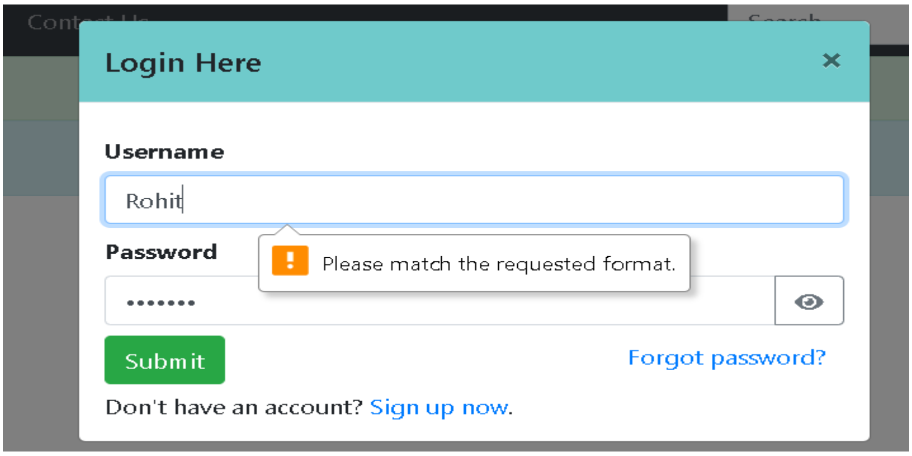



# Admin Home Page

# Test case and Test Results

## CHAPTER:5

### TC-LOGIN -01: Login with Invalid Credentials

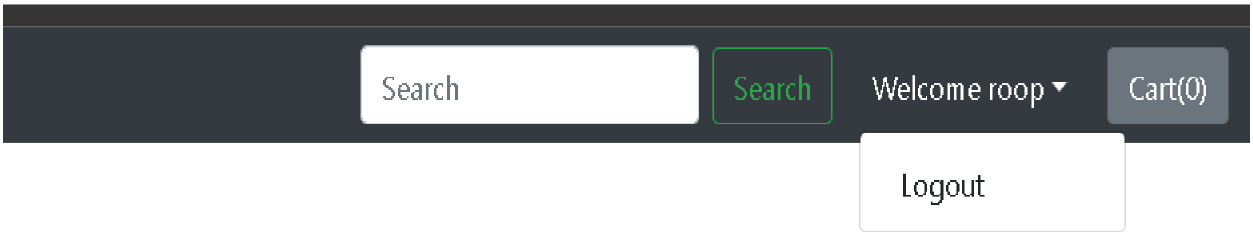



**Explanation:**

Enter wrong password or unregistered email

Display error message: "Invalid Username or Password"

### TC-LOGIN -02: Login with valid credentials

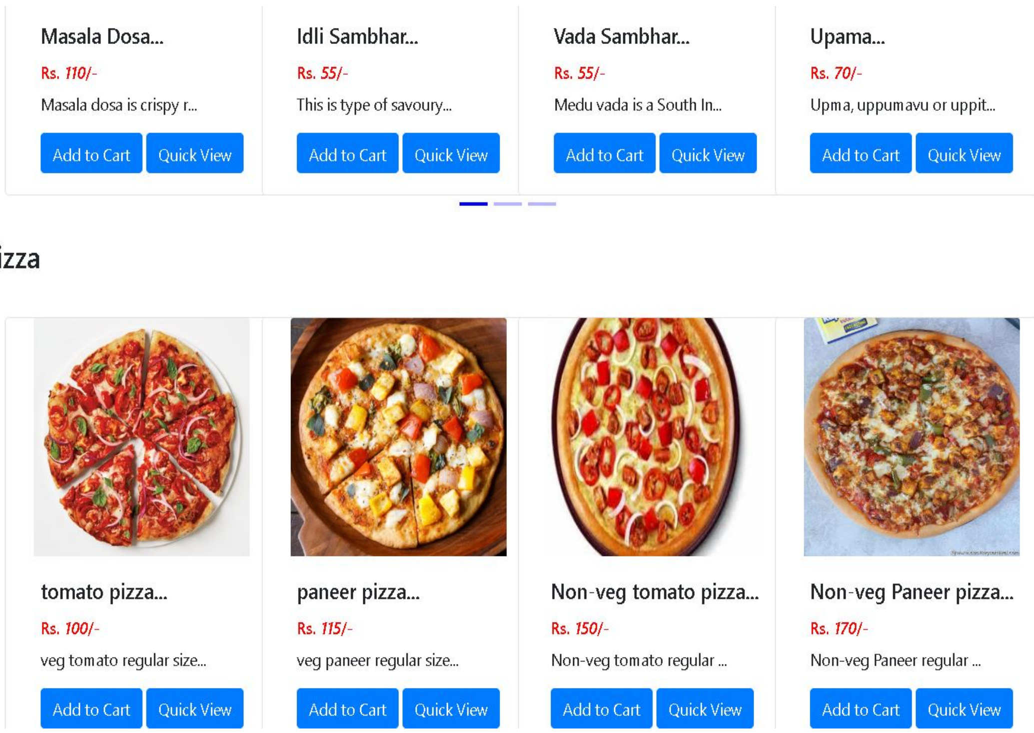



**Explanation:**

Enter registered email and correct password

User successfully logs in and show the user name on the dashboard

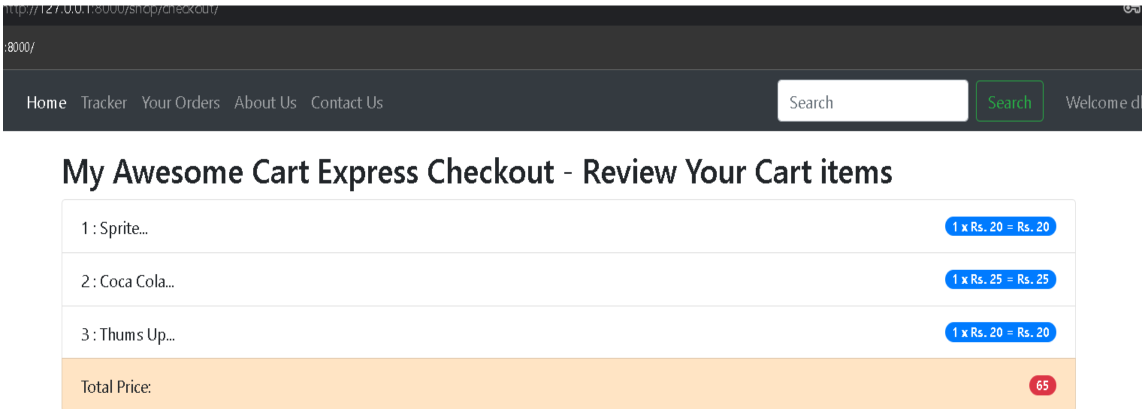
## TC-LOGIN -03: Browse and Select and Add to cart Food Items




## Explanation:

Item added to cart and displayed in order summary

## TC-LOGIN -04: Review Order

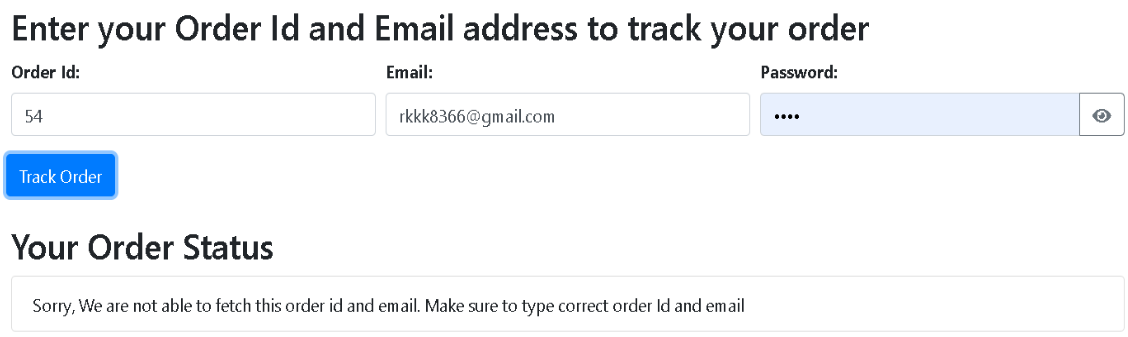



## Explanation:

Click **Review Order** before final submission

Order summary displayed with total price and selected items

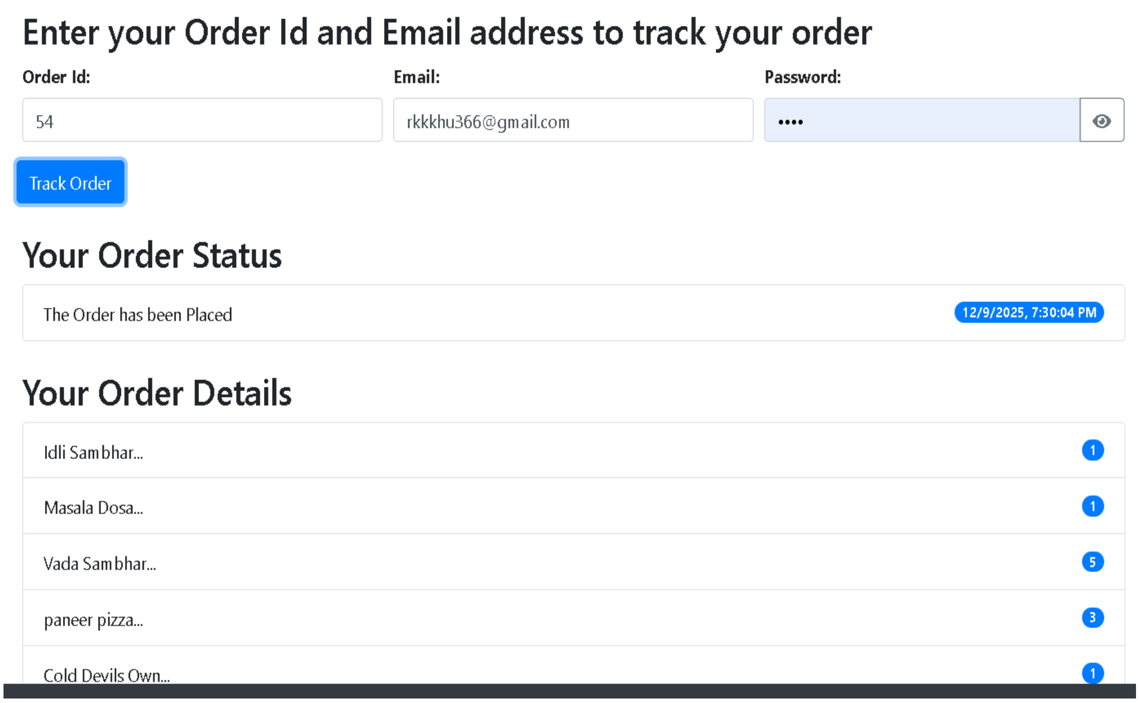
**TC-LOGIN -05:** Tracking order with include invalid order ID and username




**Explanation:**

Click Track order after that it will show your name and Order ID incorrect.

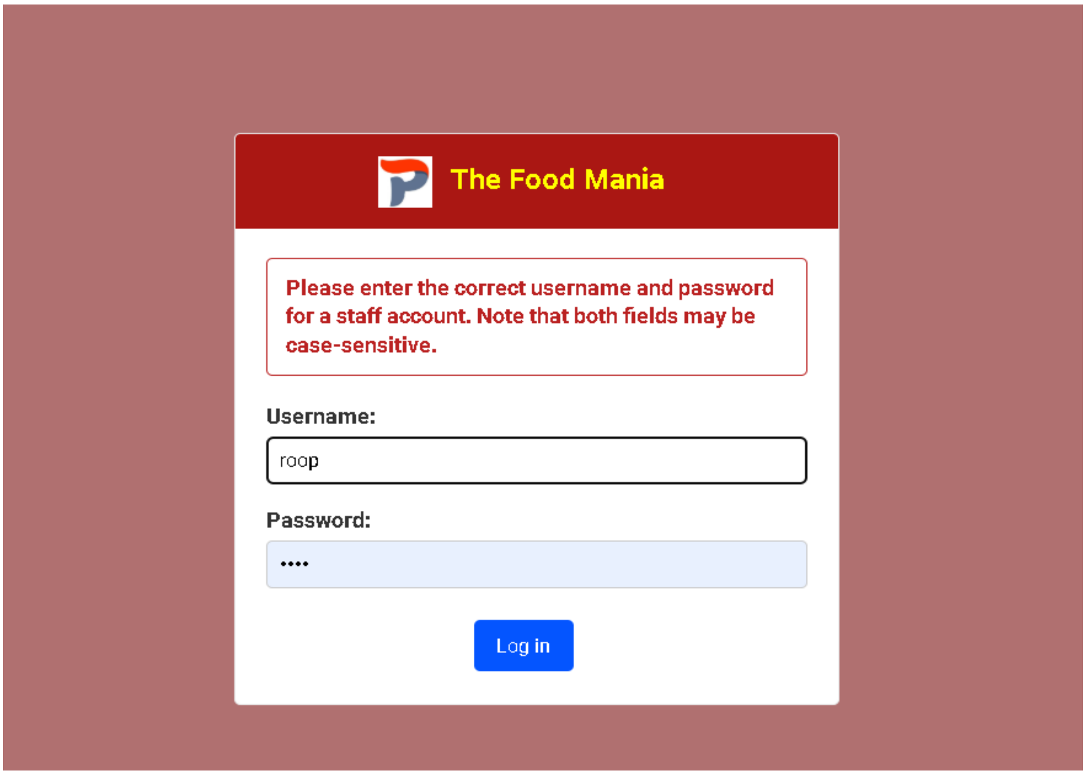
**TC-LOGIN -06:** Tracking order with include valid order ID and username




**Explanation:**

Click Track order after that it will shows as per your name and Order ID, details Of your order status.

**TC-LOGIN -07**: Admin Login



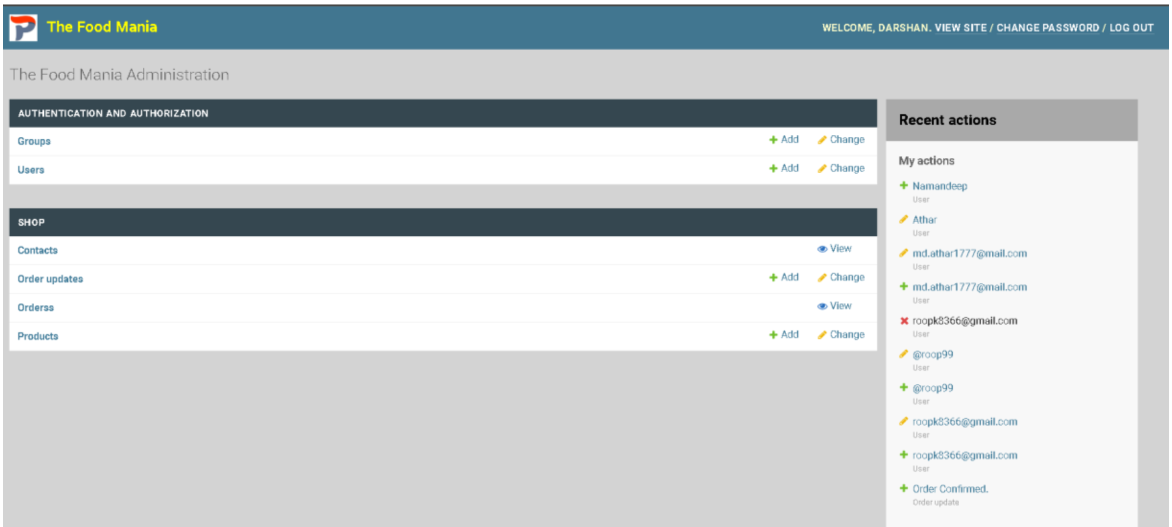

## Explanation:

Enter invalid admin credentials

Admin dashboard will show  please enter the correct username and password for a staff account. Note that both fields may be case-sensitive

.

**TC-LOGIN -08**: Admin Login Valid




## Explanation:

Enter valid admin credentials, admin dashboard opens successfully

# Conclusion:

**CHAPTER:6**

After reviewing our work, the conclusion is that after many adjustments the system works. As good as it is now, there can still be made many adjustments/improvements. However in the time was given that two persons can work on this project, the overall results are satisfactory in our opinion. The report covers the entire course of the project and results are there were needed. The first weeks the work progressed slower than expected, then the pace was increased to finish on time.

For customers, web-based ordering system can make it easier to order food without having to visit the restaurants so that customers can save time and costs. For admin, they can serve customers optimally in ordering their food and making the order report easier. Payment methods can also be done by customers through a system that is available on the web to facilitate customers in paying for their orders.

# Limitations and Future Extensions:

**CHAPTER:7**

The following section describes the work that will be implemented with future releases of the software.

- Customize orders: Allow customers to customize food orders
- Enhance User Interface by adding more user interactive features. Provide Deals and promotional Offer details to home page. Provide Recipes of the Week/Day to Home Page.
- Payment Options: Add different payment options such as PayPal, Cash, Gift Cards etc. Allow to save payment details for future use.
- Allow to process an order as a Guest
- Order Process Estimate: Provide customer a visual graphical order status
- Restaurant Locator: Allow to find and choose a nearby restaurant


- https://getbootstrap.com/
- https://www.djangoproject.com/
- https://github.com/
- http://stackoverflow.com/
- https://codewithharry.com/

# Reference :

**CHAPTER:8**

Following links and websites were referred during the development of this project:


- Laudon, K. C., & Laudon, J. P. (2020). *Management Information Systems*. Pearson Education.
- W3Schools. (2024). *HTML, CSS, JavaScript, and Python Tutorials*. Retrieved from https://www.w3schools.com
- Ahmed, M., & Rahman, S. (2021). Restaurant automation and online ordering systems: A case study. *International Journal of Computer Science and Engineering*, 9(3), 45–52.
- Singh, R. (2022). A comparative analysis of digital food ordering systems. *Journal of Information Technology and Business Innovation*, 6(1), 89–97.
- Python Software Foundation. (2024). *Python Programming Language Documentation*.
- https://www.djangoproject.com
- https://www.djangoproject.com/
- https://github.com/

# Future Scope of the Project

The **Restaurant Management System** developed as part of this project has strong potential for further enhancement and real-world deployment. In the future, the system can be upgraded with advanced features such as:

1. **Online Ordering & Home Delivery Integration**
   The system can be connected with food delivery apps and allow customers to place online orders directly.
2. **Mobile Application Support**
   Creating Android and is apps will make the system more accessible for staff and customers.
3. **AI-Based Menu Recommendations**
   The system can analyze customer preferences and suggest dishes based on previous orders or trending items.
4. **Inventory Prediction and Supplier Automation**
   It can automatically track inventory usage, predict shortages, and place supplier orders.
5. **QR-Based Menu & Digital Ordering**
   Customers can scan a QR code placed on the table to browse the menu and place orders without waiting for a waiter.
6. **Customer Loyalty & Reward System**
   Points, discounts, and membership services can be added to improve customer retention.
7. **Billing & GST Automation**
   Automated tax calculation, invoice export, and integration with accounting tools like Tally can be included.
8. **Cloud Database & Multi-Branch Support**
   Restaurants with multiple outlets can manage operations through a centralized cloud-based system.

**Future Potential:** The Restaurant Management System has significant potential for future development and real-world implementation. With further enhancements such as mobile app support, online ordering and delivery integration, AI-based recommendations, QR menu ordering, automated billing, inventory prediction, and cloud support for multi-branch restaurants, the system can evolve into a complete smart restaurant solution. These upgrades will improve efficiency, enhance customer experience, support automation, and make the system scalable for both small businesses and large restaurant chains.